

Lead technique - TP Kubernetes

A l'heure actuelle, votre API fonctionne sur render.com, ce qui est ok pour un déploiement d'un PoC mais pourrait coûter vite cher en cas de scalabilité nécessaire. Dans ce TP, nous allons déployer notre container dans un cluster Kubernetes.

Plutôt que d'utiliser un cluster managé dans le cloud, nous allons faire tourner ce cluster **directement sur votre machine** grâce au Kubernetes embarqué dans Docker Desktop. Les concepts, les commandes kubectl et les fichiers YAML sont exactement les mêmes que sur un cluster cloud : seule différence, le cluster tourne en local et tout ceci ne coûte rien.

I. Mise en place du cluster Kubernetes local

Prérequis : Docker Desktop installé et démarré

Activez Kubernetes dans Docker Desktop :

- Ouvrez Docker Desktop, puis Settings (Paramètres) → Kubernetes
- Cochez "Enable Kubernetes", puis cliquez sur "Apply & Restart"
- Patientez quelques minutes : l'indicateur Kubernetes en bas de la fenêtre doit passer au vert ("Kubernetes running")

Vérifiez ensuite depuis votre terminal que `kubectl` pointe bien sur ce cluster local :

```
kubectl config get-contexts
kubectl config use-context docker-desktop
kubectl get nodes
kubectl cluster-info
```

La commande `kubectl get nodes` doit vous retourner un seul nœud nommé `docker-desktop` avec le statut `Ready`. Si ce n'est pas le cas, attendez encore un peu ou relancez Docker Desktop.

II. Création d'un déploiement & service d'exemple

Nous allons d'abord déployer une image d'exemple, afin de valider que le cluster fonctionne et de découvrir les deux objets de base : le **Deployment** (qui fait tourner des pods) et le **Service** (qui les expose sur le réseau).

```
kubectl create deployment hello-server \
  --image=us-docker.pkg.dev/google-samples/containers/gke/hello-app:1.0

kubectl expose deployment hello-server --type=LoadBalancer --port 8080
```

Docker Desktop lors d'un expose deployment, crée un load balancer sur `localhost`.

Vérifiez-le avec :

```
kubectl get service hello-server
```

Ouvrez alors <http://localhost:8080> dans votre navigateur : vous devez voir le message "Hello, world!".

Comme nous n'avons pas de console web Google Cloud, c'est `kubectl` qui vous sert d'interface d'observation. Les commandes à connaître :

```
kubectl get all
kubectl get deployments
kubectl get pods -o wide
```

```
kubect1 get services
kubect1 describe deployment hello-server
kubect1 logs -l app=hello-server --tail=50
```

III. Modification de l'image utilisée et mise en place de health checks

Nous aimerions exécuter notre image plutôt que l'image de l'exemple.

Récupérez ensuite le yaml de votre déploiement avec la commande suivante :

```
kubect1 get deployment hello-server -o yaml > deployment.yaml
```

Le `deployment.yaml` de cette commande permet de stocker le résultat de la commande précédente dans un fichier. Plusieurs lignes doivent être supprimées de ce fichier afin de pouvoir le ré-appliquer, notamment les metadata -> annotations, metadata -> generation, metadata -> managedFields, metadata -> resourceVersion. Ces informations sont liées au déploiement à un instant T, c'est pour ceci qu'ils doivent être nettoyés.

Modifiez aussi dans ce fichier l'image docker utilisée par celle que vous avez dans le github container registry, et ajoutez un health check sur le port 3000 de votre déploiement. Il vous faudra aussi insérer les variables d'environnement comme vu dans le cours ainsi qu'un fichier de secrets comme expliqué ici :

<https://kubernetes.io/fr/docs/tasks/inject-data-application/distribute-credentials-secure/>

Créez le secret en local avant d'appliquer le déploiement, par exemple :

```
kubect1 create secret generic api-secrets \
  --from-literal=GOOGLE_APPLICATION_CREDENTIALS="..."

# ou directement depuis un fichier .env :
kubect1 create secret generic api-secrets --from-env-file=.env

kubect1 get secrets
```

Info : la section `containers` de votre `deployment.yaml` devrait ressembler à ceci :

```
containers:
  - name: mon-api
    image: mon-api:1.0
    imagePullPolicy: IfNotPresent
    ports:
      - containerPort: 3000
    env:
      - name: NODE_ENV
        value: "production"
    envFrom:
      - secretRef:
          name: api-secrets
    readinessProbe:
      httpGet:
        path: /
        port: 3000
      initialDelaySeconds: 5
```

```
    periodSeconds: 10
  livenessProbe:
    httpGet:
      path: /
      port: 3000
    initialDelaySeconds: 15
    periodSeconds: 20
```

Documentation des probes :

<https://kubernetes.io/fr/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

IV. Modification du service pour avoir le bon port

Dans l'étape précédente, le service créé utilisait une image docker qui se servait du port 8080.

Faites la même opération que précédemment pour modifier le target port de 8080 à 3000 afin que le service fonctionne de nouveau.

Info :

```
kubectl get service hello-server -o yaml > service.yaml
# modifiez targetPort: 8080 -> targetPort: 3000 et nettoyez les metadata
résiduelles
kubectl apply -f service.yaml
```

Pour rappel, `port` est le port sur lequel le service écoute (et donc, en local, le port publié sur `localhost`), tandis que `targetPort` est le port sur lequel écoute votre conteneur.

Vérifiez que tout fonctionne :

```
kubectl get service hello-server
curl http://localhost:8080/
```

Si le port 8080 est déjà utilisé sur votre machine, changez la valeur de `port` dans le service (par exemple 8081).

V. Création d'un worker standalone

Jusqu'à maintenant, le worker était lié à notre api et fonctionnait de manière dépendante. Créez un autre déploiement pour ce worker.

Afin d'avoir le bon code démarré dans chaque déploiement, il est possible de le faire de deux manières :

- Une image docker différente qui contiendra votre worker
- Une variable d'environnement qui forcera le démarrage du fichier JS server ou du fichier js lié au worker. Cette solution est plus simple à mettre en place, il suffit de modifier le npm start dans votre package.json avec "node ./app/\${SCRIPT:-server.js}". Si la variable d'environnement SCRIPT est définie, votre npm start démarrera alors le script lié.

Ce déploiement réutilise la même image locale que l'API. Contrairement à l'API, le worker n'expose aucun port : il n'a donc **pas besoin de Service**, et pas de health check HTTP.

```
kubectl apply -f worker.yaml
kubectl get pods -l app=worker
kubectl logs -f deployment/worker
```

```
# pour faire varier le nombre de workers :  
kubectl scale deployment worker --replicas=3
```

VI. Dépannage et nettoyage

Quelques commandes utiles pour investiguer :

```
kubectl describe pod <nom-du-pod> # voir les événements en bas de la sortie  
kubectl logs <nom-du-pod> --previous  
kubectl get events --sort-by=.metadata.creationTimestamp  
kubectl exec -it <nom-du-pod> -- sh # ouvrir un shell dans le conteneur
```

En fin de TP, supprimez vos ressources :

```
kubectl delete deployment hello-server worker  
kubectl delete service hello-server  
kubectl delete secret api-secrets
```

En dernier recours, Docker Desktop permet de repartir de zéro via Settings → Kubernetes → “Reset Kubernetes Cluster”. Vous pouvez également désactiver Kubernetes une fois le TP terminé pour libérer les ressources de votre machine.

Aide-mémoire kubectl : <https://kubernetes.io/fr/docs/reference/kubectl/cheatsheet/>